SQL commands needed

```
Create TABLE USER_PROFILE (
user_profile_ID INT,
PRIMARY KEY(user_profile_ID),
 major_ID INT,
FOREIGN KEY(major_ID) REFERENCES  MAJOR(major_ID) ON DELETE SET NULL,
university_ID INT,
FOREIGN KEY (university_ID) REFERENCES UNIVERSITY(university_ID) ON
DELETE CASCADE,
grad_year INT,
email VARCHAR(50),
degree_level VARCHAR(50)
);

Create TABLE MAJOR(
major_ID INT,
PRIMARY KEY(major_ID),
Major_name VARCHAR(30),
Category VARCHAR(20)
);

Create TABLE INDUSTRY(
industry_ID INT,
PRIMARY KEY(industry_ID),
industry_name VARCHAR(30)
);

Create TABLE LOCATION (
state_ID INT,
PRIMARY KEY(State_ID),
state_name VARCHAR(30)); , can insert into

Create TABLE UNIVERSITY(
university_ID INT,
PRIMARY KEY(university_ID),
university_name VARCHAR(50)
);

Create TABLE JOB_ROLE(
job_role_ID INT,
PRIMARY KEY (job_role_ID),
industry_ID INT,
FOREIGN KEY (industry_ID) REFERENCES INDUSTRY(industry_ID) ON DELETE SET
NULL,
```

```
job_title VARCHAR(30),
median_salary REAL
);

Create TABLE UNEMPLOYMENT (
year INT,
month VARCHAR(20),
state_ID INT,
FOREIGN KEY (state_ID) REFERENCES LOCATION(state_ID) ON DELETE
CASCADE,
PRIMARY KEY(year,month,state_ID),
unemployment_rate REAL
);

Create TABLE PREFERENCE_PRESET (
preset_ID INT,
PRIMARY KEY(preset_ID),
 user_profile_ID INT,
FOREIGN KEY (user_profile_ID) REFERENCES USER_PROFILE(user_profile_ID) ON
DELETE CASCADE,
 industry_ID INT,
FOREIGN KEY(industry_ID) REFERENCES INDUSTRY(industry_ID) ON DELETE
CASCADE,
state_ID INT,
FOREIGN KEY(state_ID) REFERENCES LOCATION(state_ID) ON DELETE CASCADE,
 expected_salary REAL,
 max_unemployment REAL);


Create TABLE MAJOR_JOB_PREREQUISITES (

major_ID INT,

FOREIGN KEY (major_ID) REFERENCES MAJOR(major_ID) ON DELETE CASCADE,
job_role_ID INT,

FOREIGN KEY(job_role_ID) REFERENCES JOB_ROLE(job_role_ID) ON DELETE
CASCADE,

Primary KEY(major_ID,job_role_ID));
```

Housing the buffer for importing csv

```
CREATE TABLE unemploymentRAW (
    fips_code INT,
    state_area VARCHAR(100),
    year INT,
    month VARCHAR(20),
    total_civ_non_inst_pop INT,
    total_civ_labor_force INT,
    pct_pop_in_labor_force DECIMAL(5, 2),
    total_employment INT,
    pct_employed DECIMAL(5, 2),
    total_unemployment INT,
    pct_unemployed DECIMAL(5, 2),
    PRIMARY KEY (fips_code, year, month)
);
CREATE TABLE majorRAW (
    seqId INT,
    major_ID INT PRIMARY KEY,
    major VARCHAR(255),
    major_category VARCHAR(100)
);
CREATE TABLE allagesRAW (
   id INT,
    major_code INT PRIMARY KEY,
    major VARCHAR(255),
```

```
    major_category VARCHAR(100),
    total_students INT,
    employed_count INT,
    employed_full_time_year_round INT,
    unemployed_count INT,
    unemployment_rate DECIMAL(7, 6), -- Best for rates like 0.054321
    median_salary INT
);
CREATE TABLE universityRAW (
    geo_point VARCHAR(255),
    geo_shape JSON, -- Using JSON type for the coordinate objects
    object_id INT PRIMARY KEY,
    ipeds_id INT,
    name VARCHAR(255),
    address VARCHAR(255),
    city VARCHAR(100),
    state VARCHAR(50),
    zip VARCHAR(20),
    zip4 VARCHAR(20),
    telephone VARCHAR(50),
    type_code VARCHAR(50),
    status VARCHAR(50),
    population INT,
    county VARCHAR(100),
    county_fips VARCHAR(20),
```

```
country VARCHAR(10),
latitude DOUBLE,
longitude DOUBLE,
naics_code VARCHAR(20),
naics_desc VARCHAR(255),
source VARCHAR(255),
source_date VARCHAR(50),
val_method VARCHAR(100),
val_date VARCHAR(50),
website VARCHAR(255),
st_fips VARCHAR(10),
co_fips VARCHAR(10),
sector VARCHAR(100),
level_desc VARCHAR(50),
hi_offer VARCHAR(100),
deg_grant VARCHAR(100),
locale VARCHAR(100),
close_date VARCHAR(50),
merge_id VARCHAR(50),
alias TEXT,
size_set VARCHAR(100),
inst_size INT,
pt_enroll INT,
ft_enroll INT,
tot_enroll INT,
```

```
    housing INT,
    dorm_cap INT,
    tot_emp INT,
    shelter_id VARCHAR(100)
);
CREATE TABLE jobRAW (
    company VARCHAR(255),
    education TEXT,
    experience VARCHAR(100),
    industry VARCHAR(255),
    jobdescription TEXT,
    jobid BIGINT PRIMARY KEY,
    joblocation_address TEXT,
    jobtitle VARCHAR(255),
    numberofpositions INT,
    payrate VARCHAR(255)
);
```

Insert queries to move from buffer to designated tables

Insert JOB_ROLE

```
INSERT INTO JOB_ROLE (job_role_ID, industry_ID, job_title, median_salary)
SELECT
    r.jobid,
    i.industry_ID,
    LEFT(r.jobtitle, 30),
    CASE
```

```
        WHEN REGEXP_REPLACE(r.payrate, '[^0-9.]', '') = '' THEN NULL
        ELSE CAST(REGEXP_REPLACE(r.payrate, '[^0-9.]', '') AS DECIMAL(10,2))
    END
FROM jobRAW r
LEFT JOIN INDUSTRY i ON r.industry = i.industry_name;
```

Insert LOCATION

```
INSERT INTO LOCATION (state_ID, state_name)
SELECT DISTINCT fips_code, state_area
FROM unemploymentRAW
ORDER BY state_area;
```

Insert MAJOR

```
INSERT INTO MAJOR (major_ID, Major_name, Category)
SELECT major_ID, major, major_category
FROM majorRAW;
```

Insert

```
INSERT INTO UNEMPLOYMENT (year, month, state_ID, unemployment_rate)
SELECT
    year,
    month,
    fips_code,
    pct_unemployed
FROM unemploymentRAW;
```

Insert Industry

```
INSERT INTO INDUSTRY (industry_name)

SELECT DISTINCT industry

FROM jobRAW

WHERE industry IS NOT NULL

  AND industry != 'industry';
```

Insert University

```
INSERT INTO UNIVERSITY (university_ID, university_name)

SELECT ipeds_id, LEFT(name, 50)

FROM universityRAW

WHERE ipeds_id IS NOT NULL;
```

Complex Queries:

1) High Paying Majors

```
SELECT
    m.major_category,
    m.major,
    a.median_salary
FROM majorRAW m
JOIN allagesRAW a ON m.major_ID = a.major_code
WHERE m.major_category IN (
    SELECT major_category
    FROM allagesRAW
```

```
    GROUP BY major_category
    HAVING AVG(median_salary) > (SELECT AVG(median_salary) FROM
allagesRAW)
)
ORDER BY a.median_salary DESC;
```

2) Great categories

```
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM
allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 100000;
```

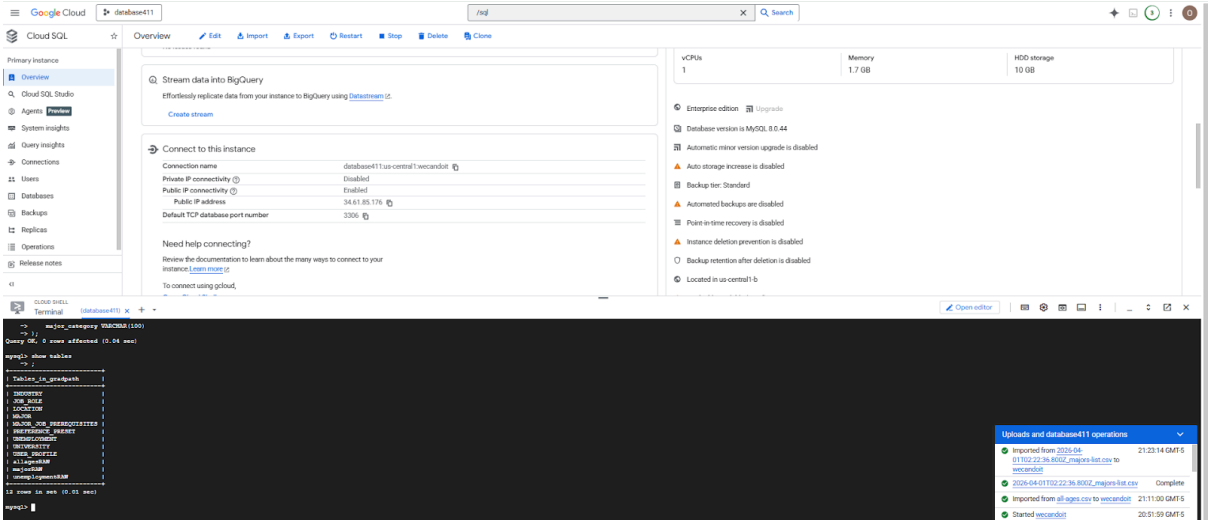
3) Unemployment on regional grads

```
SELECT
   u.state_area,
   AVG(u.total_unemployment) AS avg_state_unemployed,
   u.year
FROM unemploymentRAW u
WHERE u.year = 2023
GROUP BY u.state_area, u.year
HAVING AVG(u.total_unemployment) > (

   SELECT AVG(a.unemployed_count)
   FROM allagesRAW a
   JOIN majorRAW m ON a.major_code = m.major_ID
)
ORDER BY avg_state_unemployed DESC;
```

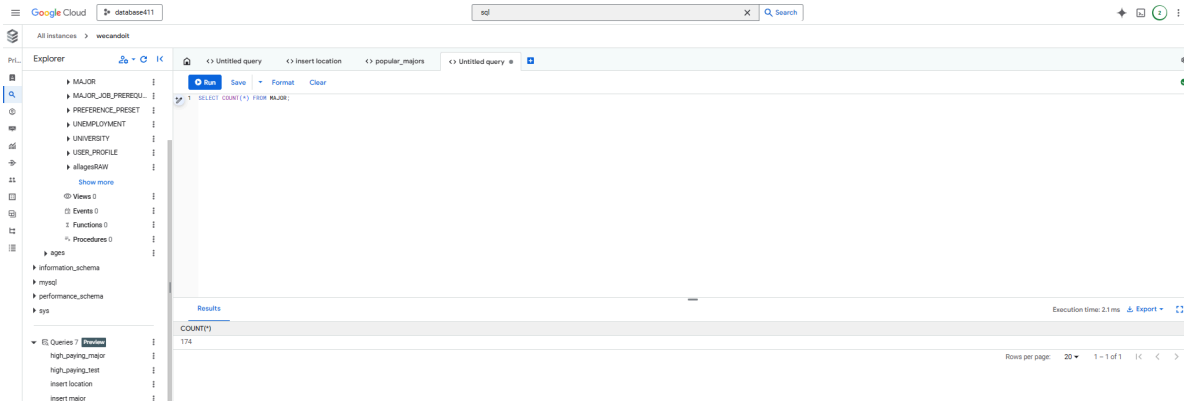
Deliverables

Screenshot connection

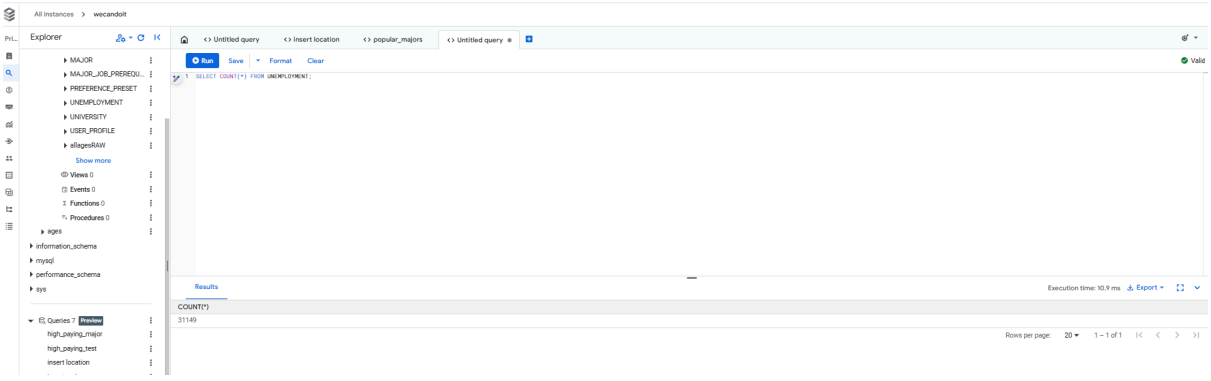


Count screenshot - 31149 rows

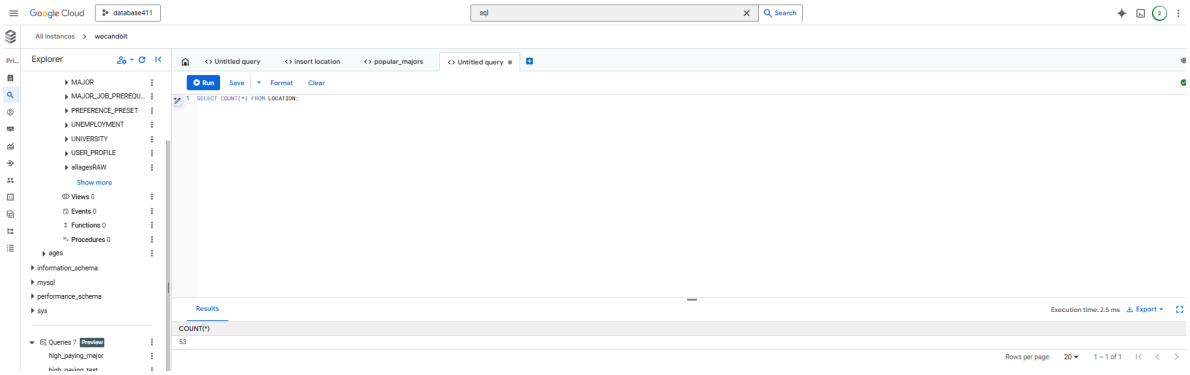
Major table - 174



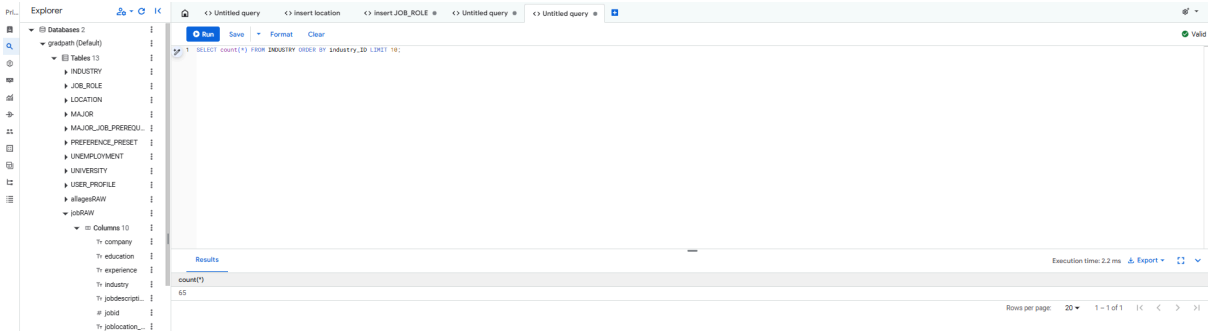
Unemployment table - 31149



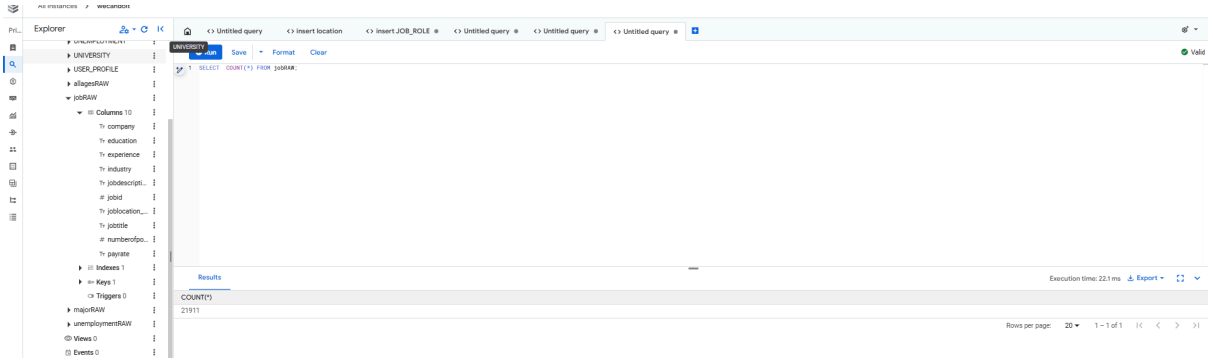
Location table - 53



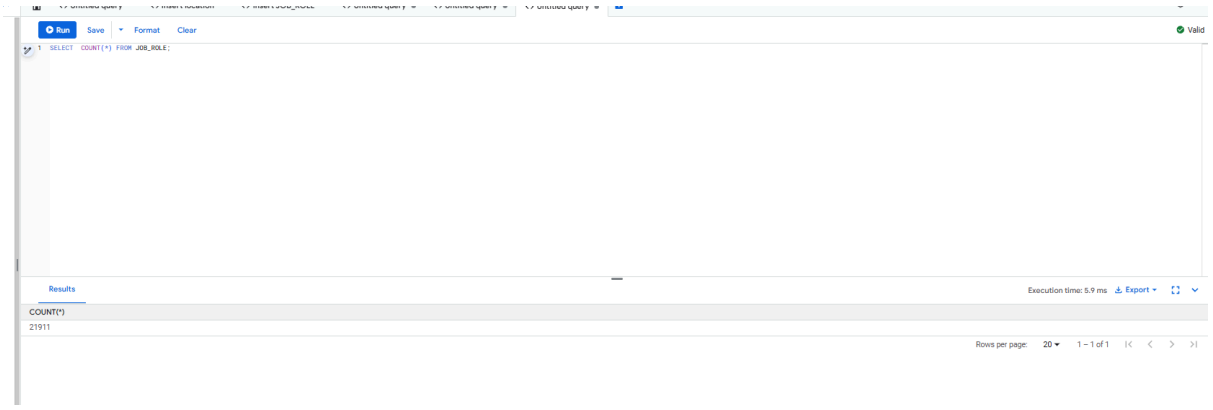
Industry -65



jobRAW - 21149

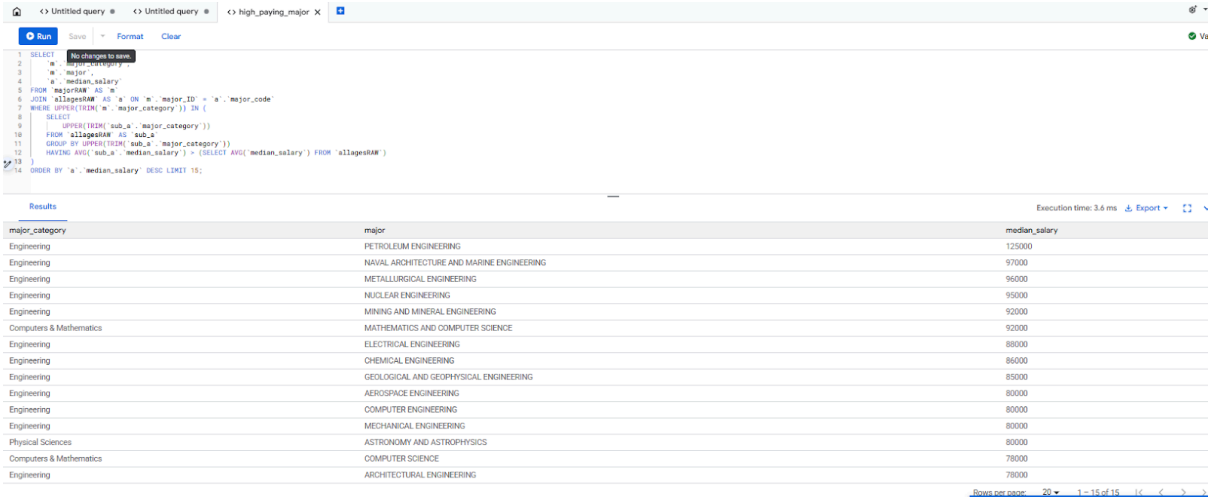


JOB_ROLE - 21149



Complex queries

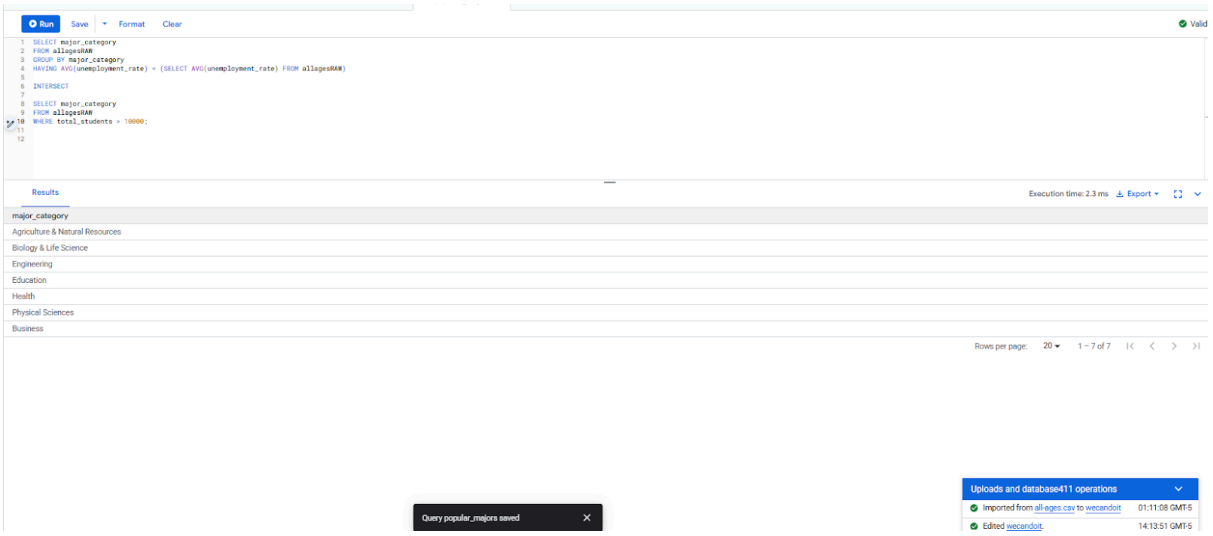
High paying majors



```
SELECT
    `m`.`major_category`,
    `m`.`major`,
    `a`.`median_salary`
FROM `majorRAW` AS `m`
JOIN `allagesRAW` AS `a` ON `m`.`major_ID` = `a`.`major_code`
WHERE UPPER(TRIM(`m`.`major_category`)) IN (
    SELECT
        UPPER(TRIM(`sub_a`.`major_category`))
    FROM `allagesRAW` AS `sub_a`
    GROUP BY UPPER(TRIM(`sub_a`.`major_category`))
    HAVING AVG(`sub_a`.`median_salary`) > (SELECT AVG(`median_salary`) FROM `allagesRAW`)
)
ORDER BY `a`.`median_salary` DESC LIMIT 15;
```

major_category	major	median_salary
Engineering	PETROLEUM ENGINEERING	125000
Engineering	NAVAL ARCHITECTURE AND MARINE ENGINEERING	97000
Engineering	METALLURGICAL ENGINEERING	96000
Engineering	NUCLEAR ENGINEERING	95000
Engineering	MINING AND MINERAL ENGINEERING	92000
Computers & Mathematics	MATHEMATICS AND COMPUTER SCIENCE	92000
Engineering	ELECTRICAL ENGINEERING	88000
Engineering	CHEMICAL ENGINEERING	86000
Engineering	GEOLOGICAL AND GEOPHYSICAL ENGINEERING	85000
Engineering	AEROSPACE ENGINEERING	80000
Engineering	COMPUTER ENGINEERING	80000
Engineering	MECHANICAL ENGINEERING	80000
Physical Sciences	ASTRONOMY AND ASTROPHYSICS	80000
Computers & Mathematics	COMPUTER SCIENCE	78000
Engineering	ARCHITECTURAL ENGINEERING	78000

Popular majors with low unemployment (<15 rows returned, only 7)



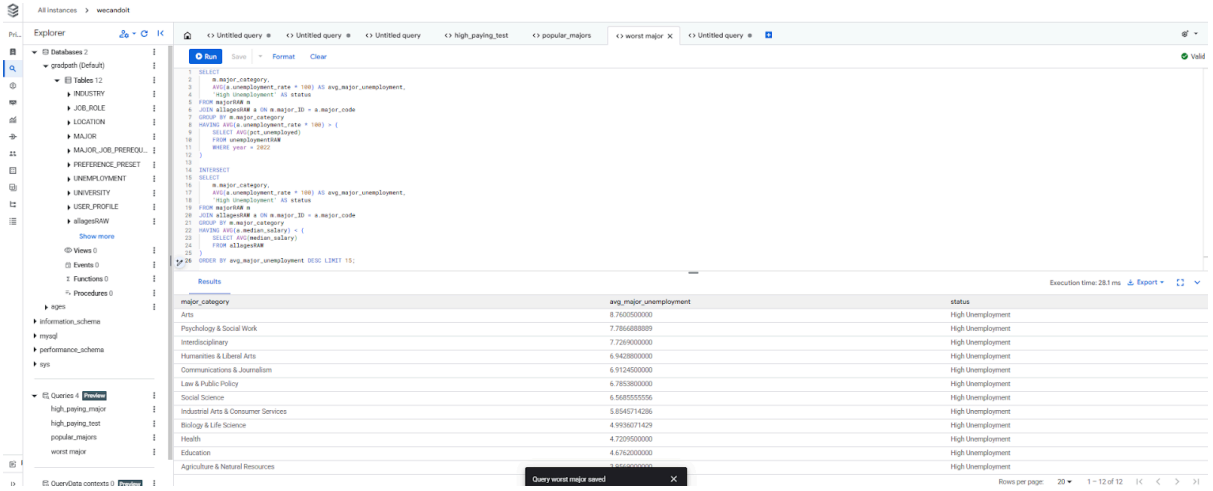
```
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 10000;
```

major_category
Agriculture & Natural Resources
Biology & Life Science
Engineering
Education
Health
Physical Sciences
Business

Worst majors 2022 (<15 rows, only 2)



```
SELECT
    m.major_category,
    AVG(a.unemployment_rate * 100) AS avg_major_unemployment,
    'High Unemployment' AS status
FROM majorRAW m
JOIN allagesRAW a ON m.major_ID = a.major_code
GROUP BY m.major_category
HAVING AVG(a.unemployment_rate * 100) > (
    SELECT AVG(pct_unemployed)
    FROM unemploymentRAW
    WHERE year = 2022
)

INTERSECT
SELECT
    m.major_category,
    AVG(a.unemployment_rate * 100) AS avg_major_unemployment,
    'High Unemployment' AS status
FROM majorRAW m
JOIN allagesRAW a ON m.major_ID = a.major_code
GROUP BY m.major_category
HAVING AVG(a.median_salary) < (
    SELECT AVG(median_salary)
    FROM allagesRAW
)
ORDER BY avg_major_unemployment DESC LIMIT 15;
```

major_category	avg_major_unemployment	status
Arts	8.7600500000	High Unemployment
Psychology & Social Work	7.7866888889	High Unemployment
Interdisciplinary	7.7269000000	High Unemployment
Humanities & Liberal Arts	6.9428800000	High Unemployment
Communications & Journalism	6.9124500000	High Unemployment
Law & Public Policy	6.7853800000	High Unemployment
Social Science	6.5685555556	High Unemployment
Industrial Arts & Consumer Services	5.8545714286	High Unemployment
Biology & Life Science	4.9936071429	High Unemployment
Health	4.7209500000	High Unemployment
Education	4.6762000000	High Unemployment
Agriculture & Natural Resources	[illegible]	High Unemployment

Optimizing Query 1: High paying major

Original query

```sql
EXPLAIN ANALYZE
SELECT
   m.major_category,
   m.major,
   a.median_salary
FROM majorRAW m
JOIN allagesRAW a
   ON m.major_ID = a.major_code
WHERE m.major_category IN (
   SELECT major_category
   FROM allagesRAW
   GROUP BY major_category
   HAVING AVG(median_salary) > (
       SELECT AVG(median_salary)
       FROM allagesRAW
   )
)
ORDER BY a.median_salary DESC;
```

Run Save Format Clear

```sql
EXPLAIN ANALYZE
SELECT
    m.major_category,
    m.major,
    a.median_salary
FROM majorRAW m
JOIN allagesRAW a
    ON m.major_ID = a.major_code
WHERE m.major_category IN (
    SELECT major_category
    FROM allagesRAW
    GROUP BY major_category
    HAVING AVG(median_salary) > (
        SELECT AVG(median_salary)
        FROM allagesRAW
    )
)
ORDER BY a.median_salary DESC;
```

Results — Execution time: 12.0 ms — Export

EXPLAIN

-> Sort: a.median_salary DESC (actual time=3.06..3.06 rows=63 loops=1) -> Stream results (cost=78.6 rows=174) (actual time=2.59..3.01 rows=63 loops=1) -> Nested loop inner join (cost=78.6 rows=174) (actual time=2.58..2.97 rows=63 loops=1) -> Filter: <in_optimizer>(m.major_category,m.major_category in (select #2)) (cost=17.7 rows=174) (actual time=2.56..2.82 rows=63 loops=1) -> Table scan on m (cost=17.7 rows=174) (actual time=0.193..0.296 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Filter: ((m.major_category = `<materialized_subquery>`.major_category)) (cost=0..0 rows=0) (actual time=0.0539..0.0539 rows=0.205 loops=44) -> Limit: 1 row(s) (cost=0..0 rows=0) (actual time=0.0536..0.0536 rows=0.205 loops=44) -> Index lookup on <materialized_subquery> using <auto_distinct_key> (major_category=m.major_category) (actual time=0.0534..0.0534 rows=0.205 loops=44) -> Materialize with deduplication (cost=0..0 rows=0) (actual time=2.31..2.31 rows=4 loops=1) -> Filter: (avg(allagesRAW.median_salary) > (select #3)) (actual time=2.28..2.29 rows=4 loops=1) -> Table scan on <temporary> (actual time=0.49..0.499 rows=17 loops=1) -> Aggregate using temporary table (actual time=0.488..0.488 rows=17 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0305..0.166 rows=174 loops=1) -> Select #3 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.median_salary) (cost=35.6 rows=1) (actual time=1.75..1.75 rows=1 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0293..0.137 rows=174 loops=1) -> Single-row index lookup on a using PRIMARY (major_code=m.major_ID) (cost=0.251 rows=1) (actual time=0.00205..0.00208 rows=1 loops=63)

Original Cost analysis:

The EXPLAIN ANALYZE result shows that the query finally returns 63 rows, but MySQL still performs several expensive operations. It scans all 174 rows from majorRAW, materializes the subquery into 4 rows in about 2.31 ms, and uses a temporary table to aggregate 17 rows. The plan also shows two scans on allagesRAW, each reading 174 rows, and a final sort on a.median_salary DESC that takes about 3.06 ms. Although the join on a is efficient because it uses a primary-key lookup, the main cost still comes from full scans, subquery materialization, temporary aggregation, and sorting.

Design 1 : Index on grouping/filter column

The join itself is efficient, so the main optimization opportunities are reducing unnecessary scans and improving grouping and sorting support through indexing.

Queries:

```sql
CREATE INDEX idx_allagesraw_major_category
ON allagesRAW(major_category);
```

Run Save Format Clear

```sql
EXPLAIN ANALYZE
SELECT
    m.major_category,
    m.major,
    a.median_salary
FROM majorRAW m
JOIN allagesRAW a
    ON m.major_ID = a.major_code
WHERE m.major_category IN (
    SELECT major_category
    FROM allagesRAW
    GROUP BY major_category
    HAVING AVG(median_salary) > (
        SELECT AVG(median_salary)
        FROM allagesRAW
    )
)
ORDER BY a.median_salary DESC;
```

Results — Execution time: 4.0 ms — Export

EXPLAIN

-> Sort: a.median_salary DESC (actual time=0.755..0.759 rows=63 loops=1) -> Stream results (cost=78.6 rows=174) (actual time=0.47..0.708 rows=63 loops=1) -> Nested loop inner join (cost=78.6 rows=174) (actual time=0.468..0.682 rows=63 loops=1) -> Filter: <in_optimizer>(m.major_category,m.major_category in (select #2)) (cost=17.7 rows=174) (actual time=0.46..0.607 rows=63 loops=1) -> Table scan on m (cost=17.7 rows=174) (actual time=0.0316..0.0906 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Filter: ((m.major_category = `<materialized_subquery>`.major_category)) (cost=37.4..37.4 rows=1) (actual time=0.0101..0.0101 rows=0.205 loops=44) -> Limit: 1 row(s) (cost=37.3..37.3 rows=1) (actual time=0.00993..0.00993 rows=0.205 loops=44) -> Index lookup on <materialized_subquery> using <auto_distinct_key> (major_category=m.major_category) (actual time=0.00979..0.00979 rows=0.205 loops=44) -> Materialize with deduplication (cost=37.3..37.3 rows=17) (actual time=0.406..0.406 rows=4 loops=1) -> Filter: (avg(allagesRAW.median_salary) > (select #3)) (cost=35.6 rows=17) (actual time=0.19..0.395 rows=4 loops=1) -> Group aggregate: avg(allagesRAW.median_salary) (cost=35.6 rows=17) (actual time=0.0553..0.312 rows=17 loops=1) -> Index scan on allagesRAW using idx_allagesraw_major_category (cost=18.2 rows=174) (actual time=0.0407..0.241 rows=174 loops=1) -> Select #3 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.median_salary) (cost=35.6 rows=1) (actual time=0.0698..0.0699 rows=1 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0111..0.0542 rows=174 loops=1) -> Single-row index lookup on a using PRIMARY (major_code=m.major_ID) (cost=0.251 rows=1) (actual time=978e-6..0.00101 rows=1 loops=63)

After adding idx_allagesraw_major_category, MySQL uses an index scan on allagesRAW in the subquery instead of direct scanning. This means the index is effective for the grouping column major_category. However, the overall plan still keeps full scan on majorRAW (174 rows), materializes the subquery into 4 rows, aggregates 17 rows with a temporary table, and sorts the final 63-row result. So, Design 1 helps the grouping step but not help the main query to filter matching responses from the subquery.

Design 2 : Index on the salary column

This design targets the median_salary column, which is used in two important parts of the query. It may help improve sorting on median_salary and also support aggregation-related access patterns. However, it may not fully eliminate Using filesort, because the query also involves joins and subqueries.
Queries:

```
CREATE INDEX idx_allagesraw_median_salary
ON allagesRAW(median_salary);
```

Run Save Format Clear

```
EXPLAIN ANALYZE
SELECT
    m.major_category,
    m.major,
    a.median_salary
FROM majorRAW m
JOIN allagesRAW a
    ON m.major_ID = a.major_code
WHERE m.major_category IN (
    SELECT major_category
    FROM allagesRAW
    GROUP BY major_category
    HAVING AVG(median_salary) > (
        SELECT AVG(median_salary)
        FROM allagesRAW
    )
)
ORDER BY a.median_salary DESC;
```

Results Execution time: 2.8 ms Export

EXPLAIN

-> Sort: a.median_salary DESC (actual time=0.801..0.806 rows=63 loops=1) -> Stream results (cost=78.6 rows=174) (actual time=0.512..0.773 rows=63 loops=1) -> Nested loop inner join (cost=78.6 rows=174) (actual time=0.509..0.749 rows=63 loops=1) -> Filter: <in_optimizer>(m.major_category,m.major_category in (select #2)) (cost=17.7 rows=174) (actual time=0.496..0.647 rows=63 loops=1) -> Table scan on m (cost=17.7 rows=174) (actual time=0.0342..0.098 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Filter: ((m.major_category = `<materialized_subquery>`.major_category)) (cost=0..0 rows=0) (actual time=0.0108..0.0108 rows=0.205 loops=44) -> Limit: 1 row(s) (cost=0..0 rows=0) (actual time=0.0106..0.0106 rows=0.205 loops=44) -> Index lookup on <materialized_subquery> using <auto_distinct_key> (major_category=m.major_category) (actual time=0.0104..0.0104 rows=0.205 loops=44) -> Materialize with deduplication (cost=0..0 rows=0) (actual time=0.435..0.435 rows=4 loops=1) -> Filter: (avg(allagesRAW.median_salary) > (select #3)) (actual time=0.424..0.429 rows=4 loops=1) -> Table scan on <temporary> (actual time=0.323..0.325 rows=17 loops=1) -> Aggregate using temporary table (actual time=0.321..0.321 rows=17 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0503..0.183 rows=174 loops=1) -> Select #3 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.median_salary) (cost=35.6 rows=1) (actual time=0.0846..0.0847 rows=1 loops=1) -> Covering index scan on allagesRAW using idx_allagesraw_median_salary (cost=18.2 rows=174) (actual time=0.0315..0.0663 rows=174 loops=1) -> Single-row index lookup on a using PRIMARY (major_code=m.major_ID) (cost=0.251 rows=1) (actual time=0.00139..0.00141 rows=1 loops=63)

After adding idx_allagesraw_median_salary, MySQL uses a covering index scan on allagesRAW in one subquery step instead of direct scanning. This shows that the salary index is effective for salary-related access. However, the overall plan still keeps full scan on majorRAW (174 rows), materializes the subquery into 4 rows, aggregates 17 rows with a temporary table, and sorts the final 63-row result. Hence, design 2 brought little improvements to the query.

Design 3 : Composite index on join column and sorting column

This design combines major_code and median_salary into one composite index. It is intended to support access to allagesRAW after join matching and may be more useful than a single-column salary index. However, since the query still includes grouping and sorting, it may not fully eliminate the expensive operations.

```
CREATE INDEX idx_allagesraw_majorcode_salary
ON allagesRAW(major_code, median_salary);
```

Run Save Format Clear Vali

```
EXPLAIN ANALYZE
SELECT
    m.major_category,
    m.major,
    a.median_salary
FROM majorRAW m
JOIN allagesRAW a
    ON m.major_ID = a.major_code
WHERE m.major_category IN (
    SELECT major_category
    FROM allagesRAW
    GROUP BY major_category
    HAVING AVG(median_salary) > (
        SELECT AVG(median_salary)
        FROM allagesRAW
    )
)
ORDER BY a.median_salary DESC;
```

Results Execution time: 3.0 ms Export

EXPLAIN

```
-> Sort: a.median_salary DESC (actual time=0.717..0.722 rows=63 loops=1) -> Stream results (cost=78.6 rows=174) (actual time=0.376..0.69 rows=63 loops=1) -> Nested loop inner join (cost=78.6 rows=174) (actual time=0.374..0.605 rows=63
loops=1) -> Filter: <in_optimizer>(m.major_category,m.major_category in (select #2)) (cost=17.7 rows=174) (actual time=0.363..0.511 rows=63 loops=1) -> Table scan on m (cost=17.7 rows=174) (actual time=0.0315..0.0916 rows=174 loops=1) ->
Select #2 (subquery in condition; run only once) -> Filter: ((m.major_category = `<materialized_subquery>`.major_category)) (cost=0..0 rows=0) (actual time=0.0079..0.0079 rows=0.205 loops=44) -> Limit: 1 row(s) (cost=0..0 rows=0) (actual
time=0.0077..0.0077 rows=0.205 loops=44) -> Index lookup on <materialized_subquery> using <auto_distinct_key> (major_category=m.major_category) (actual time=0.00756..0.00756 rows=0.205 loops=44) -> Materialize with deduplication (cost=0..0
rows=0) (actual time=0.31..0.31 rows=4 loops=1) -> Filter: (avg(allagesRAW.median_salary) > (select #3)) (actual time=0.299..0.305 rows=4 loops=1) -> Table scan on <temporary> (actual time=0.222..0.225 rows=17 loops=1) -> Aggregate using
temporary table (actual time=0.221..0.221 rows=17 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0163..0.0852 rows=174 loops=1) -> Select #3 (subquery in condition; run only once) -> Aggregate:
avg(allagesRAW.median_salary) (cost=35.6 rows=1) (actual time=0.0633..0.0634 rows=1 loops=1) -> Covering index scan on allagesRAW using idx_allagesraw_majorcode_salary (cost=18.2 rows=174) (actual time=0.0122..0.0466 rows=174 loops=1)
-> Single-row index lookup on a using PRIMARY (major_code=m.major_ID) (cost=0.251 rows=1) (actual time=0.00126..0.00129 rows=1 loops=63)
```

After adding Design 3, the plan is better than the original one because MySQL can use the composite index idx_allagesraw_majorcode_salary in the subquery. This reduces some unnecessary work on allagesRAW and makes execution faster. However, the query still has several expensive steps, including scanning majorRAW, materializing the subquery, using a temporary table for aggregation, and sorting the final output. So this design improves the query. Thus we choose design 3 to optimize the query.

Optimizing Query 2: popular majors

```
EXPLAIN ANALYZE
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 10000;
```

Run Save Format Clear

```
EXPLAIN ANALYZE
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 10000;
```

Results Execution time: 3.1 ms Export

EXPLAIN

-> Table scan on <intersect temporary> (cost=20.6..20.6 rows=0) (actual time=0.637..0.638 rows=7 loops=1) -> Intersect materialize with deduplication (cost=18.1..18.1 rows=0) (actual time=0.636..0.636 rows=8 loops=1) -> Filter: (??? < (select #2)) (actual time=0.441..0.449 rows=8 loops=1) -> Table scan on <temporary> (actual time=0.316..0.319 rows=17 loops=1) -> Aggregate using temporary table (actual time=0.314..0.314 rows=17 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0454..0.131 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.unemployment_rate) (cost=35.6 rows=1) (actual time=0.108..0.108 rows=1 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0113..0.0624 rows=174 loops=1) -> Filter: (allagesRAW.total_students > 10000) (cost=18.1 rows=58) (actual time=0.0171..0.0886 rows=154 loops=1) -> Table scan on allagesRAW (cost=18.1 rows=174) (actual time=0.0146..0.0654 rows=174 loops=1)

In the current execution query, MySQL performs three separate scans on allagesRAW, and each one reads the full 174 rows. These scans are used for three different purposes: grouping by major_category, computing the overall AVG(unemployment_rate), and filtering rows with total_students > 10000.
Besides the repeated scans, the query also creates extra overhead through intermediate processing. The grouping step uses a temporary table and produces 17 grouped rows, and the INTERSECT part is materialized with deduplication, generating 8 rows before the final output of 7 rows. Taken together, these steps show that the main performance problem is not a join, but the repeated table access and the extra work needed for grouping and intersection processing. Therefore, the most useful optimization direction is to add indexes that better support grouping, aggregation, and filtering on allagesRAW.

Design 1 Index on the grouping column

This design focuses on the GROUP BY major_category part. Since the first half of the query groups rows by major_category, indexing this column may help MySQL organize grouped rows more efficiently.

Run Save Format Clear

```
EXPLAIN ANALYZE
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 10000;
```

Results Execution time: 2.4 ms Export

EXPLAIN

-> Table scan on <intersect temporary> (cost=55.6..58.1 rows=17) (actual time=0.628..0.63 rows=7 loops=1) -> Intersect materialize with deduplication (cost=55.4..55.4 rows=17) (actual time=0.627..0.627 rows=8 loops=1) -> Filter: (avg(allagesRAW.unemployment_rate) < (select #2)) (cost=35.6 rows=17) (actual time=0.189..0.451 rows=8 loops=1) -> Group aggregate: avg(allagesRAW.unemployment_rate) (cost=35.6 rows=17) (actual time=0.1..0.358 rows=17 loops=1) -> Index scan on allagesRAW using idx_allagesraw_major_category (cost=18.2 rows=174) (actual time=0.0766..0.262 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.unemployment_rate) (cost=35.6 rows=1) (actual time=0.0795..0.0796 rows=1 loops=1) -> Table scan on allagesRAW (cost=18.2 rows=174) (actual time=0.0161..0.0599 rows=174 loops=1) -> Filter: (allagesRAW.total_students > 10000) (cost=18.1 rows=58) (actual time=0.014..0.0793 rows=154 loops=1) -> Table scan on allagesRAW (cost=18.1 rows=174) (actual time=0.0124..0.0631 rows=174 loops=1)

Design 1 adds an index on major_category, which turns out to be a negative optimization because it does not reduce the actual amount of data processed. In the EXPLAIN ANALYZE output, MySQL still performs an index scan with 174 rows on allagesRAW, meaning it is effectively reading the whole table even after using idx_allagesraw_major_category. The grouped aggregation remains expensive at cost=35.6, and the scalar subquery SELECT AVG(unemployment_rate) FROM allagesRAW still requires a table scan of 174 rows. In addition, the second half of the INTERSECT still scans the table with cost=18.1 rows=174 before filtering total_students > 1000. Because the major bottlenecks are still full-table aggregation and intersection processing, the final estimated cost rises to 55.6..56.8, which is much higher than the better plan's 20.6..20.6. This shows that indexing only the grouping column helps ordering slightly, but does not address the real cost drivers of the query.

Design 2 Cover the filter branch

Design 2 focuses on the second branch of the query by indexing total_students together with major_category, so MySQL can filter rows with total_students > 10000 more efficiently while also returning major_category directly from the index.

Design 2 decreases performance compared with the original query. Its total estimated cost rises from 20.6..20.6 to 35.5..35.5, so the optimizer considers it more expensive. Although the index is used in the second branch, that branch still returns 154 rows, so the filtering effect is limited. Meanwhile, the first branch still performs a table scan of 174 rows and temporary-table aggregation, so the main bottleneck is not removed.

Design 3 Reverse the column order for the first branch

This design tests an alternative column order by indexing unemployment_rate before major_category, in order to check whether MySQL benefits more from starting with the aggregated column rather than the grouping column.

Result of optimization 2

Although several indexing designs were tested, the improvements were limited or negative because allagesRAW is a very small table, and the main cost comes from repeated scans, aggregation, and INTERSECT materialization rather than indexable point/range lookups.

Run Save Format Clear

```sql
EXPLAIN ANALYZE
SELECT major_category
FROM allagesRAW
GROUP BY major_category
HAVING AVG(unemployment_rate) < (SELECT AVG(unemployment_rate) FROM allagesRAW)

INTERSECT

SELECT major_category
FROM allagesRAW
WHERE total_students > 10000;
```

Results Execution time: 2.4 ms Export

EXPLAIN

-> Table scan on <intersect temporary> (cost=20.6..20.6 rows=0) (actual time=0.664..0.665 rows=7 loops=1) -> Intersect materialize with deduplication (cost=18.1..18.1 rows=0) (actual time=0.663..0.663 rows=8 loops=1) -> Filter: (??? < (select #2)) (actual time=0.447..0.459 rows=8 loops=1) -> Table scan on <temporary> (actual time=0.334..0.338 rows=17 loops=1) -> Aggregate using temporary table (actual time=0.332..0.332 rows=17 loops=1) -> Covering index scan on allagesRAW using idx_allagesraw_unemployment_majorcategory (cost=18.2 rows=174) (actual time=0.0411..0.115 rows=174 loops=1) -> Select #2 (subquery in condition; run only once) -> Aggregate: avg(allagesRAW.unemployment_rate) (cost=35.6 rows=1) (actual time=0.094..0.0941 rows=1 loops=1) -> Covering index scan on allagesRAW using idx_allagesraw_unemployment_majorcategory (cost=18.2 rows=174) (actual time=0.0123..0.065 rows=174 loops=1) -> Filter: (allagesRAW.total_students > 10000) (cost=18.1 rows=58) (actual time=0.0269..0.104 rows=154 loops=1) -> Table scan on allagesRAW (cost=18.1 rows=174) (actual time=0.0242..0.0863 rows=174 loops=1)

Design 3 is about the same as the original query, with no clear improvement in estimated performance. Both plans have a total cost of about 20.6..20.6, so changing the index order to put unemployment_rate before major_category did not make the optimizer choose a cheaper plan.

Overall result :
Query 2 is likely showing negative optimization because the table is very small, with only 174 rows. For such a small dataset, a full table scan is often cheaper than using an index. In this query, the main cost also comes from aggregation, the scalar subquery, and INTERSECT deduplication rather than row lookup alone. As a result, adding indexes does not significantly reduce the workload and may even introduce extra overhead, making the indexed plans look worse than the original.

Optimizing Query 3:

```
CLOUD SHELL
Terminal (database411) ×                                                    Open Editor

Gemini CLI is available in Cloud Shell terminal! Type gemini to try it. Learn more

| performance_schema |
| sys                |
+--------------------+
6 rows in set (0.01 sec)

mysql> use gradpath;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> explain analyze SELECT
    ->     m.major_category,
    ->     AVG(a.unemployment_rate * 100) AS avg_major_unemployment,
    ->     'High Unemployment' AS status
    -> FROM majorRAW m
    -> JOIN allagesRAW a ON m.major_ID = a.major_code
    -> GROUP BY m.major_category
    -> HAVING AVG(a.unemployment_rate * 100) > (
    ->     SELECT AVG(pct_unemployed)
    ->     FROM unemploymentRAW
    ->     WHERE year = 2022
    -> )
    -> INTERSECT
    -> SELECT
    ->     m.major_category,
    ->     AVG(a.unemployment_rate * 100) AS avg_major_unemployment,
    ->     'High Unemployment' AS status
    -> FROM majorRAW m
    -> JOIN allagesRAW a ON m.major_ID = a.major_code
    -> GROUP BY m.major_category
    -> HAVING AVG(a.median_salary) < (
    ->     SELECT AVG(median_salary)
    ->     FROM allagesRAW
    -> )
    -> ORDER BY avg_major_unemployment DESC LIMIT 15;
+-----------------------
```

Result

```
| -> Limit: 15 row(s)  (cost=2.6..2.6 rows=0) (actual time=10.9..10.9 rows=12 loops=1)
    -> Sort: avg_major_unemployment DESC, limit input to 15 row(s) per chunk  (cost=2.6..2.6 rows=0) (actual time=10.9..10.9 rows=12 loops=1)
        -> Table scan on <intersect temporary>  (cost=2.5..2.5 rows=0) (actual time=10.9..10.9 rows=12 loops=1)
            -> Intersect materialize with deduplication  (cost=0..0 rows=0) (actual time=10.9..10.9 rows=16 loops=1)
                -> Filter: (??? > (select #2))  (actual time=10..10.1 rows=16 loops=1)
                    -> Table scan on <temporary>  (actual time=0.562..0.571 rows=17 loops=1)
                        -> Aggregate using temporary table  (actual time=0.561..0.561 rows=17 loops=1)
                            -> Nested loop inner join  (cost=78.6 rows=174) (actual time=0.0457..0.328 rows=174 loops=1)
                                -> Table scan on m  (cost=17.7 rows=174) (actual time=0.0285..0.0833 rows=174 loops=1)
                                -> Single-row index lookup on a using PRIMARY (major_code=m.major_ID)  (cost=0.251 rows=1) (actual time=0.00119..0.00121 rows=1 loops=174)
                    -> Select #2 (subquery in condition; run only once)
                        -> Aggregate: avg(unemploymentRAW.pct_unemployed)  (cost=281 rows=1) (actual time=9.41..9.41 rows=1 loops=1)
                            -> Index lookup on unemploymentRAW using unemployment_year (year=2022)  (cost=218 rows=624) (actual time=0.427..9.23 rows=624 loops=1)
                -> Filter: (??? < (select #4))  (actual time=0.657..0.675 rows=13 loops=1)
                    -> Table scan on <temporary>  (actual time=0.563..0.569 rows=17 loops=1)
                        -> Aggregate using temporary table  (actual time=0.562..0.562 rows=17 loops=1)
                            -> Nested loop inner join  (cost=78.6 rows=174) (actual time=0.0509..0.356 rows=174 loops=1)
                                -> Table scan on m  (cost=17.7 rows=174) (actual time=0.0386..0.0942 rows=174 loops=1)
                                -> Single-row index lookup on a using PRIMARY (major_code=m.major_ID)  (cost=0.251 rows=1) (actual time=0.00115..0.00118 rows=1 loops=174)
                    -> Select #4 (subquery in condition; run only once)
                        -> Aggregate: avg(allagesRAW.median_salary)  (cost=35.6 rows=1) (actual time=0.0769..0.077 rows=1 loops=1)
                            -> Table scan on allagesRAW  (cost=18.2 rows=174) (actual time=0.0155..0.0592 rows=174 loops=1)

 |
+----------------------
```

Final design conclusion: The best final index is the first design unemploymentRAW(year) because it targets the only large table in the query and directly addresses the WHERE clause filter, greatly reducing the cost.